39

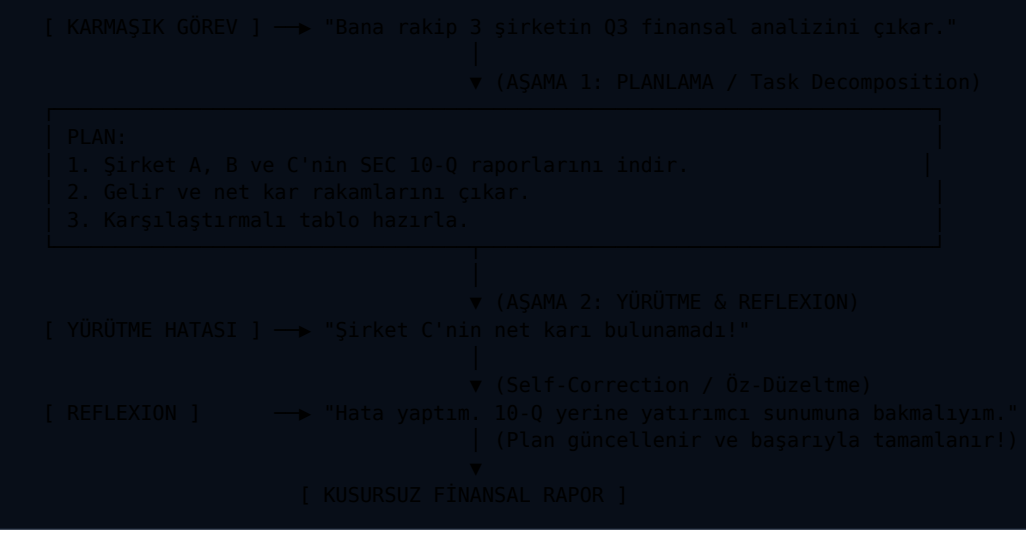
BÖLÜM: AI AJANLARI (AI AGENTS), OTONOM AKIŞLAR VE BELLEK MIMARISI | MODULE: AI AGENTS, AUTONOMOUS LOOPS & CONVERSATIONAL MEMORY SYSTEMS

Planlama ve Görev Ayrıştırma: Plan-and-Solve, Reflexion & Self-Correction

Planning & Task Decomposition: Plan-and-Solve, Reflexion & Self-Correction Loops

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir satranç ustası hemen elini tahtaya uzatıp taş oynamaz. Önce durur, 5 hamle sonrasını kafasında planlar: 'Atı buraya koyarsam, o piyonu sürer, ben de vezirle şah çekerim' (Plan-and-Solve). Eğer rakip beklemediği bir hamle yaparsa 'Eyvah, planım bozuldu, hemen yeni bir strateji kurmalıyım' der ve planını revize eder (Reflexion / Self-Correction). Basit yapay zeka hemen cevaba atlar; akıllı bir yapay zeka ajanı ise önce plan yapar, hata yaparsa hatasından ders çıkarıp kendini düzeltir!

🇬🇧 ENGLISH (ELI5)

A chess grandmaster never reaches blindly across the board. They mentally simulate five moves ahead: 'If I advance my knight, they will counter with a pawn push, opening my bishop for checkmate' (Plan-and-Solve). When an unexpected blunder occurs, they reflect critically, analyze the tactical error, and pivot to a defensive counter-strategy (Reflexion / Self-Correction). Naive LLMs answer impulsively; advanced agents draft a structured multi-step plan, self-critique intermediate outputs, and correct mistakes autonomously!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Planlama Mimarisi Algoritmaları: 1) Plan-and-Solve (Wang et al., 2023): Görevi tek seferde çözmek yerine önce bağımsız alt adımlara (Sub-tasks DAG) böler. Adımlar birbirine bağımlıysa (Dependency Graph) sırayla, bağımsızsa Java 25 Virtual Threads ile paralel yürütülür. 2) Reflexion (Shinn et al., 2023): Ajan her eylemden sonra bir 'Öz-Eleştiri' (Self-Critique) adımı çalıştırır: 'Ürettiğim SQL geçerli mi? Çıktı beklenen şemaya uyuyor mu?' Eğer uymuyorsa hata logunu kendi hafızasına ekleyip eylemi yeniden dener. 3) LLM Compiler: Planlama ile yürütmeyi paralel task scheduler ile optimize eder.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Advanced Agentic Planning Topologies: 1) Plan-and-Solve Paradigm: Explicitly decouples macro-planning from execution. Synthesizes a Directed Acyclic Graph (DAG) of discrete sub-tasks. Independent DAG nodes execute concurrently across Java 25 Virtual Threads. 2) Reflexion Architecture (Shinn et al.): Augments agents with dynamic memory and self-reflective feedback loops. If code generation fails a unit test, the execution trace is injected back into the prompt scratchpad, triggering targeted self-correction. 3) Deterministic Rollback: State machines track DAG state; failed sub-steps can be retried without restarting the entire pipeline.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25: Plan-and-Solve Görev Ayrıştırma ve Plan Modeli
import java.util.List;

public class TaskPlannerEngine {
    public record SubTask(int stepOrder, String description, boolean isCompleted) {}
    public record ExecutionPlan(String overallGoal, List<SubTask> subTasks) {}

    public static ExecutionPlan createPlanForQuery(String userGoal) {
        // Modelin ürettiği DAG planı simülasyonu
        var task1 = new SubTask(1, "Postgres veritabanından son 30 günün satışlarını çek", true);
        var task2 = new SubTask(2, "Müşteri iade oranlarını hesapla", true);
        var task3 = new SubTask(3, "Grafik için özet JSON oluştur", false);

        return new ExecutionPlan(userGoal, List.of(task1, task2, task3));
    }
}

void main() {
    var plan = TaskPlannerEngine.createPlanForQuery("Aylık E-Ticaret Sağlık Raporu");
    IO.println("Hedef: " + plan.overallGoal());
    for (var task : plan.subTasks()) {
        IO.println("Adım " + task.stepOrder() + ": " + task.description() + " [Tamamlandı: " +
task.isCompleted() + "]");
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Mülakat Uyarısı: Reflexion ve Self-Correction döngülerine de mutlaka `maxRetries = 3` sınırı konulmalıdır. Eğer model 3 denemede de hatayı düzeltemiyorsa inatla devam edip token yakmamalı, hemen 'Hata çözülemedi, insan müdahalesi gerekiyor' diyerek durmalıdır.

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Reflexion loop hazard: A model hallucinating an impossible solution will self-reflect into an infinite loop if unconstrained. Enforce strict `max_reflections \<= 3` bounds and fail-fast with human escalation if errors persist.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Plan-and-Solve, karmaşık problemleri yönetilebilir alt görevlere (Sub-tasks) böler.

• Reflexion (Öz-Düzeltme), başarısız olan adımları analiz edip hatadan ders çıkarır.

• Planlama adımları Java 25 sanal iş parçacıklarıyla asenkron ve paralel çalıştırılabilir.

🎯 Key Takeaways for the Interview

• Plan-and-Solve decomposes complex requests into an executable DAG of sub-tasks.

• Reflexion enables agents to learn from intermediate execution errors dynamically.

• Independent plan steps can be scheduled concurrently using Java 25 Virtual Threads.